PROJECT ACTIVITY REPORT

Activity 1.2: Install Required Libraries and Set Up Virtual Environment

Project: Multimodal Attendance & Engagement Agent

Objective

Set up a dedicated Python development environment for the Multimodal Attendance & Engagement Agent. A virtual environment isolates project dependencies from the system-wide Python installation, preventing version conflicts and providing a stable, portable and reproducible workspace. All required libraries are installed from a single `requirements.txt` and verified through successful imports and a successful backend server launch.

Section 01 — Introduction

Modern AI applications depend on many third-party libraries covering computer vision, speech recognition, natural language processing, deep learning and backend development. Managing these dependencies carefully is essential so that the application behaves consistently on every developer machine and on the deployment server.

A project-specific virtual environment keeps packages private to the project, which simplifies development, testing and deployment and follows the practice standard in professional AI and cloud engineering teams.

Section 02 — Approach

The environment is built with Python's built-in `venv` module. After activation, every dependency is installed from one `requirements.txt`, so all contributors receive the same package set. The installation is then validated by importing the essential libraries and launching the FastAPI backend.

1. ENVIRONMENT INIT	2. ACTIVATION & PIP	3. DEPENDENCIES	4. VALIDATION
Python check & venv	Isolated shell & upgrade	Batch package install	Imports & FastAPI run
`python -m venv venv`	`pip install --upgrade`	`-r requirements.txt`	`uvicorn app:app`

Figure 1: Four-stage workflow for environment isolation, dependency installation and server validation.

Section 03 — Process

Step 1: Verify Python installation

Confirm that Python is installed and that its version is compatible with the project (Python 3.9 – 3.12 recommended for PyTorch and OpenCV wheels).

```
python --version
```

Step 2: Create the virtual environment

A virtual environment named `venv` is created in the project directory. It contains its own interpreter and package folder.

```
python -m venv venv
```

Step 3: Activate the virtual environment

Shell	Command
Windows Command Prompt	`venv\Scripts\activate`
Windows PowerShell	`.\venv\Scripts\Activate.ps1`
Linux / macOS	`source venv/bin/activate`

The prompt shows `(venv)` once the environment is active. If PowerShell blocks the script, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.

Step 4: Upgrade pip

Upgrading pip, setuptools and wheel improves compatibility with modern packages and reduces build errors.

```
python -m pip install --upgrade pip setuptools wheel
```

Step 5: Install required libraries

Dependencies are declared in `requirements.txt`. The list below is a reference baseline for the project's capability areas; adjust it to the modules you actually use.

```
# requirements.txt  (reference baseline)

# Computer vision / face detection
opencv-python>=4.8,<5
numpy>=1.24
# Deep learning / NLP
torch>=2.1
torchvision>=0.16
transformers>=4.38
# Data processing
pandas>=2.0
# Backend API
fastapi>=0.110
uvicorn[standard]>=0.27
websockets>=12.0
python-multipart>=0.0.9
pydantic>=2.5
python-dotenv>=1.0
# Database connectivity
SQLAlchemy>=2.0
```

```
pip install -r requirements.txt
```

Capability area	Libraries
Computer vision, face detection	OpenCV, NumPy
Deep learning models	PyTorch, torchvision
Speech processing / NLP	Transformers (pretrained speech and language models)
Data processing	Pandas, NumPy
Backend API development	FastAPI, Uvicorn, WebSockets, Pydantic
Database connectivity	SQLAlchemy

Note: PyTorch builds differ for CPU and CUDA GPUs. For a GPU machine, take the exact install command from the selector on pytorch.org before running the step above.

Step 6: Verify installed packages

```
pip list
```

Step 7: Verify library imports

The script below imports each essential library and prints its version, so a failure points directly to the missing package.

```
# verify_env.py
import importlib, sys

LIBS = ["cv2", "numpy", "pandas", "torch", "transformers", "fastapi"]
failed = []
for name in LIBS:
    try:
        mod = importlib.import_module(name)
        print(f"[OK]   {name:<13} {getattr(mod, '__version__', '')}")
    except ImportError:
        print(f"[FAIL] {name} is not installed")
        failed.append(name)

if failed:
    sys.exit(1)
print("All required libraries imported successfully.")
```

```
python verify_env.py
```

Step 8: Launch the backend server

Starting the server confirms that the dependencies work together. Run it from the folder containing `app.py`.

```
uvicorn app:app --reload
```

A successful start shows `Uvicorn running on http://127.0.0.1:8000`. Open `/docs` to see the interactive API documentation.

Section 04 — Technical Details

Component	Technology used
Programming language	Python 3.x
Virtual environment	Python venv
Package manager	pip
Dependency file	requirements.txt
Backend framework	FastAPI (served by Uvicorn)
AI libraries	PyTorch, Transformers
Computer vision	OpenCV
Data processing	NumPy, Pandas

Section 05 — Advantages of Using a Virtual Environment

- Prevents conflicts between different Python projects.
- Keeps project dependencies isolated from the system installation.
- Ensures reproducibility across multiple development systems.
- Simplifies deployment and collaboration.
- Allows different projects to use different library versions.
- Improves project maintainability and stability.
- Follows industry-standard software engineering practices.

Troubleshooting

Symptom	Cause	Resolution
ModuleNotFoundError: No module named 'cv2'	venv not active or package missing.	Activate venv, then pip install opencv-python.
Activation script blocked (PowerShell)	Restrictive execution policy.	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, or use activate.bat.
torch install is very slow or fails	Large wheel, unsupported Python version or CPU/GPU mismatch.	Use Python 3.9 - 3.12 and the command generated on pytorch.org.
Error loading app:app	Wrong folder or module name.	Run uvicorn from the folder that contains app.py.
Port 8000 in use	Another process is using the port.	uvicorn app:app --reload --port 8001

Best practices

- Keep `venv/` and `.env` in `.gitignore`; commit only requirements.txt.
- After a successful setup, run `pip freeze > requirements.lock.txt` to pin exact versions for deployment.

- Test library upgrades in a temporary venv before changing the project's dependency file.

Section 06 — Validation

The environment setup is validated through the following checks:

Check	Command	Status
Python installation verified	python --version	[ ]
Virtual environment created	python -m venv venv	[ ]
Virtual environment activated	(venv) prompt visible	[ ]
Installed packages verified	pip list	[ ]
Core libraries imported	python verify_env.py	[ ]
Backend server launched	uvicorn app:app --reload	[ ]

Tick the Status column after running each check on your machine.